Energy fluxes and stability across a natural stream temperature gradient

Nate Heili

2025-11-19

Using energetic food web models to assess ecological stability

"Food webs and their models are at best caricatures of reality, and thus have both the truth and the falsity of caricature." May 1973

Ecologists have long sought to understand how ecosystems respond to disturbances. Theory and empirical work suggest that **interaction strengths** within complex communities can reveal how a system responds to perturbation. Specifically, communities with a high frequency of weak interactions can dampen the effects of species loss following a disturbance. One way to quantify species interactions is by estimating energy fluxes through the food web, which can then be used to calculate stability metrics.

In this tutorial, I use empirical data from a broad stream thermal gradient (Junker et al. 2024) to construct Jacobian matrices of interaction strengths using the *fluxweb* package (Moore and de Ruiter 2012; Gauzens et al. 2019). Check out *https://cran.r-project.org/web/packages/fluxweb/vignettes/How_to.pdf* for the package vignette.

```r
# Clean environemnt
rm(list = ls())

# load packages
library(fluxweb)
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## Warning: package 'dplyr' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(readr)
library(ggridges)
```

Load data

Data for the bioenergetic model (from Junker et al.) are stored on my GitHub page *https://github.com/nheili42/stability-tutorial*. The metadata file explaining the contents of each data source is on Github.

```
# Create local data folder
dir.create("data", showWarnings = FALSE)

# Annual temperature in kT
url <- "https://github.com/nheili42/stability-tutorial/raw/main/data/annual_tempkt.rds"
dest <- "data/annual_tempkt.rds"
download.file(url, dest, mode = "wb")
annual_tempkt <- readRDS(dest)

# Seasonal stream temperatures
url <- "https://github.com/nheili42/stability-tutorial/raw/main/data/stream_temps_full.rds"
dest <- "data/stream_temps_full.rds"
download.file(url, dest, mode = "wb")
stream_temps_full <- readRDS(dest)

# Taxa list
url <- "https://github.com/nheili42/stability-tutorial/raw/main/data/taxaDf.rds"
dest <- "data/taxaDf.rds"
download.file(url, dest, mode = "wb")
taxaDf <- readRDS(dest)

# Community structure / production
url <- "https://github.com/nheili42/stability-tutorial/raw/main/data/prod_full_fin.rds"
dest <- "data/prod_full_fin.rds"
download.file(url, dest, mode = "wb")
prod_full_fin <- readRDS(dest)

# Diet matrices
url <- "https://github.com/nheili42/stability-tutorial/raw/main/data/diet_matrices.rds"
dest <- "data/diet_matrices.rds"
download.file(url, dest, mode = "wb")
diet_matrices <- readRDS(dest)
```

Data wrangling

To estimate energy fluxes using the *fluxing* function, we need the community structure (consumer biomasses)(mg/m), trophic interactions (in this case, weighted by consumer preference quantified via gut content analysis), assimilation efficiencies (based on prey type), and the energetic demand of each consumer. Energy loss to metabolism uses the MTE which incorporates sp. body mass (mg/ind) and environmental temperature. I'll describe more detail about the energy losses vector later.

Because each input for *fluxing* needs to be a vector, I wrangle the data into a nested list for each stream. Eventually, I will reorganize this into one nested list.

Community structure

I organize the community biomass (mg/m2) and individual body mass (mg/ind.) for each stream. The data is currently stored as bootstrap estimates, so I take the mean value for each taxon on a given date and site. I estimate individual body mass (mg/ind) by dividing population biomass (mg/m2) by abundance (ind/m2). I include basal resources here as they will need to be apart of the "community" for the fluxing function later.

In Jochum et al. 2021, they assign basal resources 1 g/m2 and body mass as 1g/ind. I set them as NA for now and may adjust later.

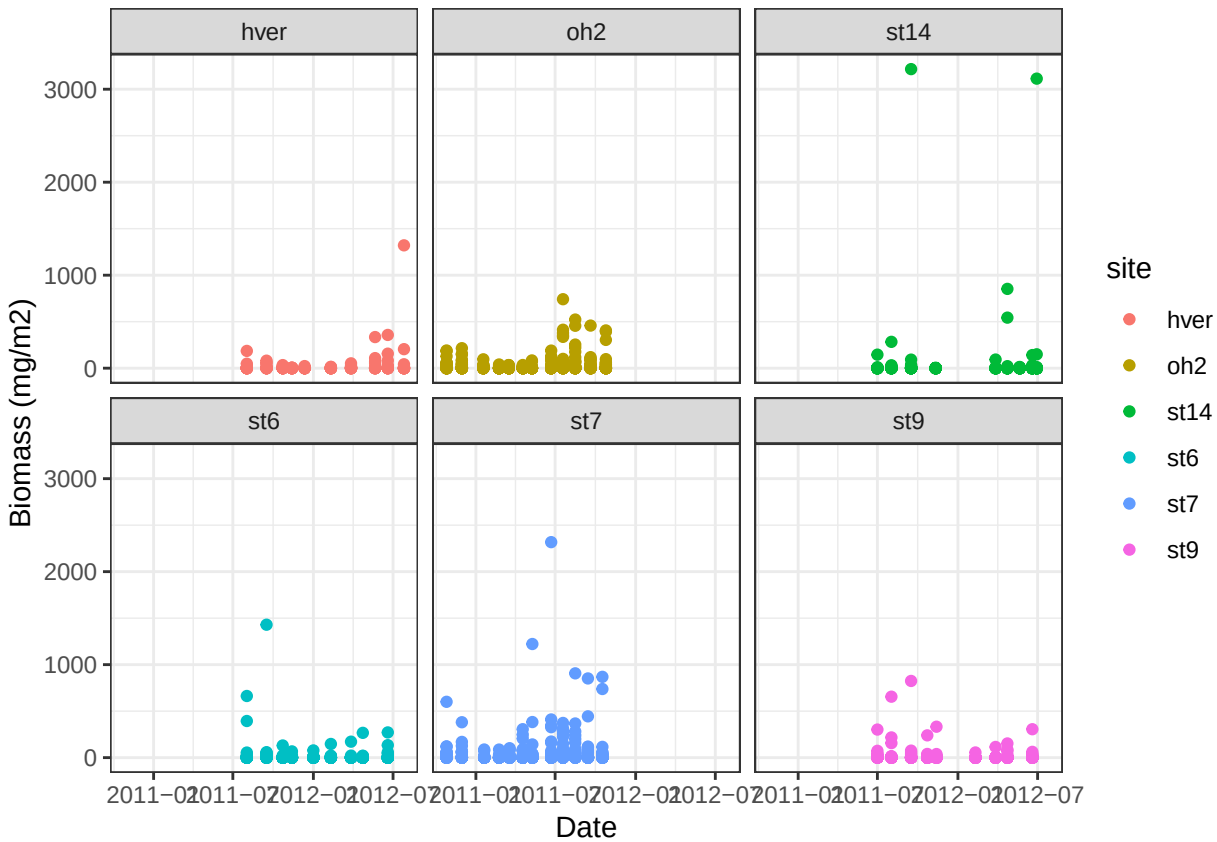
```r
# Bootstrap estimates of consumer biomass (mg/m2) on a given date
bio_full <- map_dfr(
  prod_full_fin,
  ~ .x[["Bintboots.full"]],
  .id = "site"
)

# Average biomass from bootstrap
bio_filtered <- bio_full %>%
  group_by(site, taxon_id, date_id) %>%
  summarise(mean_bio_mgm2 = mean(bio_mg_m),
            mean_bio_gm2 = mean(bio_mg_m)/1000)
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by site, taxon_id, and date_id.
## i Output is grouped by site and taxon_id.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(site, taxon_id, date_id))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```r
# Bootstrap estimates of consumer densities (ind/m2) on a given date
abundance_full <- map_dfr(
  prod_full_fin,
  ~ .x[["Nintboots.full"]],
  .id = "site"
)

# Average abundance from bootstrap
abundance_filtered <- abundance_full %>%
  group_by(site, taxon_id, date_id) %>%
  summarise(mean_abund_ind_m2 = mean(n_ind_m))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by site, taxon_id, and date_id.
## i Output is grouped by site and taxon_id.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(site, taxon_id, date_id))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```r
# Combine abundance and biomass to one df
community <- left_join(bio_filtered,
                       abundance_filtered,
                       by = join_by(site, taxon_id, date_id))

community$date_id <- ymd(community$date_id)

# visualize biomass through time to understand temporal scales
ggplot(community, aes(x = date_id, y = mean_bio_mgm2, color = site)) +
  geom_point() +
  facet_wrap(~site, nrow = 2, ncol = 3) +
  theme_bw() +
  labs(x = "Date",
       y = "Biomass (mg/m2)")
```



Fixing taxon lists

I noticed errors (spelling and/or lumping errors) that need to be addressed so the diet matrix matches the community structure data. I found this out later on in the script, but have put it here in order to summarize bootstraps and diet matrices correctly to have the same species list. The length of community data (biomasses, metabolic rates, body size) need to match the diet matrix in order for **fluxing** to work.

```
# Lumping or spelling changes for community data
name_cleaner <- tribble(
  ~old_name, ~final_name,
  "Lumb 3", "Lumbricidae",
  "Oligochaeta A", "Lumbricidae",
  "Oligochaeta B", "Lumbricidae",
  "Oligochaeta C", "Lumbricidae",
  "Naidid 1", "Naidid",
  "Naidid 2", "Naidid",
  "Tub. 2", "Naidid",
  "Tubificid 1", "Naidid",
  "Ceratopogonidae A", "Ceratopegonid",
  "Ceratopogonidae B", "Ceratopegonid",
  "Eukiefferiella", "Eukiefferiella sp.",
  " Eukiefferiella", "Eukiefferiella sp.",
  "Eukiefferiella claripennis", "Eukiefferiella sp.",
  "Eukiefferiella minor", "Eukiefferiella sp.",
  "Orthocladius fridgidus", "Orthocladius frigidus",
  "S. vernum pupa", "Simulium vernum",
  "S. vitattum pupa", "Simulium vittatum",
```

```r
  "Oribatid 1", "Oribatid",
  "Oribatid 2", "Oribatid",
  "Oribatid 3", "Oribatid"
)

# Create cleaned df with names that match diet data
community <- community %>%

  # Join with cleaned names
  left_join(name_cleaner, by = c("taxon_id" = "old_name")) %>%

  # Update taxon_id; if it wasn't in the table, keep the original
  mutate(taxon_id = coalesce(final_name, taxon_id)) %>%



  # Re-sum values in case of lumping
  # Group by site, taxon, and date to combine lumped names
  group_by(site, taxon_id, date_id) %>%
  summarise(
    across(starts_with("mean_"), sum, .names = "{.col}"),
    .groups = "drop"
  )
```

There are differences in temporal sampling among sites in the gradient. I am estimating annual energy fluxes (to start). Therefore, I will summarize annually and keep in mind that oh2 and st7 are 2010-2011 and others are 2011-2012.

```r
# Estimate individual biomass of each species (g/ind)
community$g_ind <- community$mean_bio_gm2 / community$mean_abund_ind_m2
community$mg_ind <- community$g_ind * 1000

# Add basal resources to the community dataframe. Need these for fluxing to match the trophic interacti
basal_taxa <- c(
  "amorphous_detritus",
  "animal",
  "cyanobacteria",
  "diatom",
  "filamentous",
  "green_algae",
  "plant_material"
)

# According to Gauzens et al 2019 and Jochum et al. 2021, assign basal resources 1 g/m2 and 1 g/ind
# I opt to use virtually 0
basal_df <- community %>%
  ungroup() %>%
  distinct(site, date_id) %>%
  crossing(taxon_id = basal_taxa) %>%
  mutate(mean_bio_gm2 = 0,
         mean_abund_ind_m2 = 0,
         g_ind = 0,
         mg_ind = 0)
```

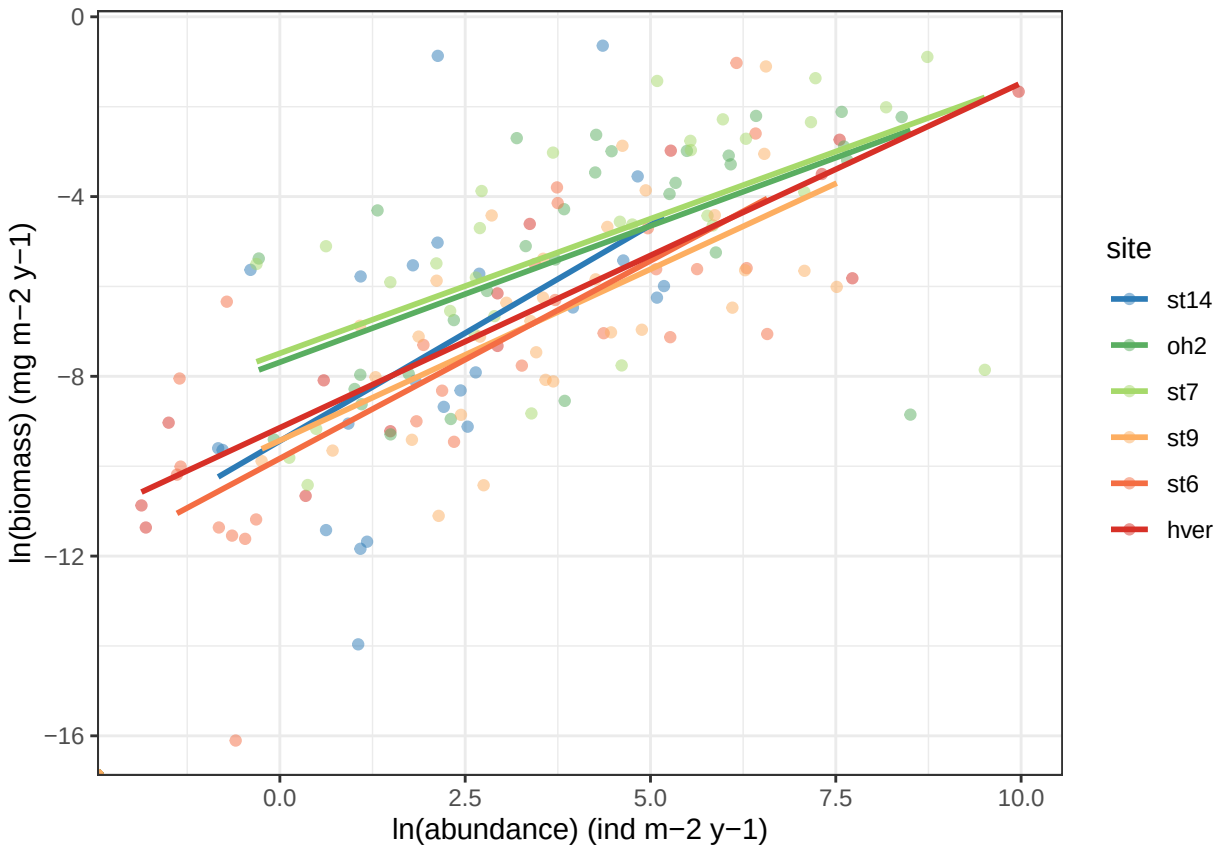
```r
# Add basal resources to community df for each site on a given date
community_seasonal <- bind_rows(community, basal_df) %>%
  arrange(site, date_id) %>%
  mutate(taxon_id = str_trim(taxon_id)) # removes white space from start/end of taxon_id names

# Summarize community at annual scale (or the sampling period)
community_annual <- community_seasonal %>%
  group_by(site, taxon_id) %>%
  summarise(
    sampling_period = paste(year(min(date_id)), year(max(date_id)), sep = "-"),
    bio_gm2y = mean(mean_bio_gm2),
    abun_indm2y = mean(mean_abund_ind_m2),
    .groups = "drop") %>%
  mutate(g_indy = bio_gm2y/abun_indm2y,
         mg_indy = g_indy * 1000)
```

###Visualize annual community data

```r
# Order sites by increasing temperature for plotting
community_annual <- community_annual %>%
  mutate(site = factor(site,
                       levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
                       ordered = TRUE))

# site colors for plotting
site_colors <- c(
  st14 = "#2C7BB6",  # deep blue (coldest)
  oh2  = "#5AAE61",  # blue-green
  st7  = "#A6D96A",  # green
  st9  = "#FDAE61",  # orange
  st6  = "#F46D43",  # orange-red
  hver = "#D73027"   # red (warmest)
)


# Quick check of abundance-biomass relationship
ggplot(community_annual, aes(x=log(abun_indm2y), y=log(bio_gm2y), color = site)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = lm, , se = F) +
  scale_color_manual(values = site_colors)+
  theme_bw() +
  labs(x = "ln(abundance) (ind m-2 y-1)",
       y = "ln(biomass) (mg m-2 y-1)")
```

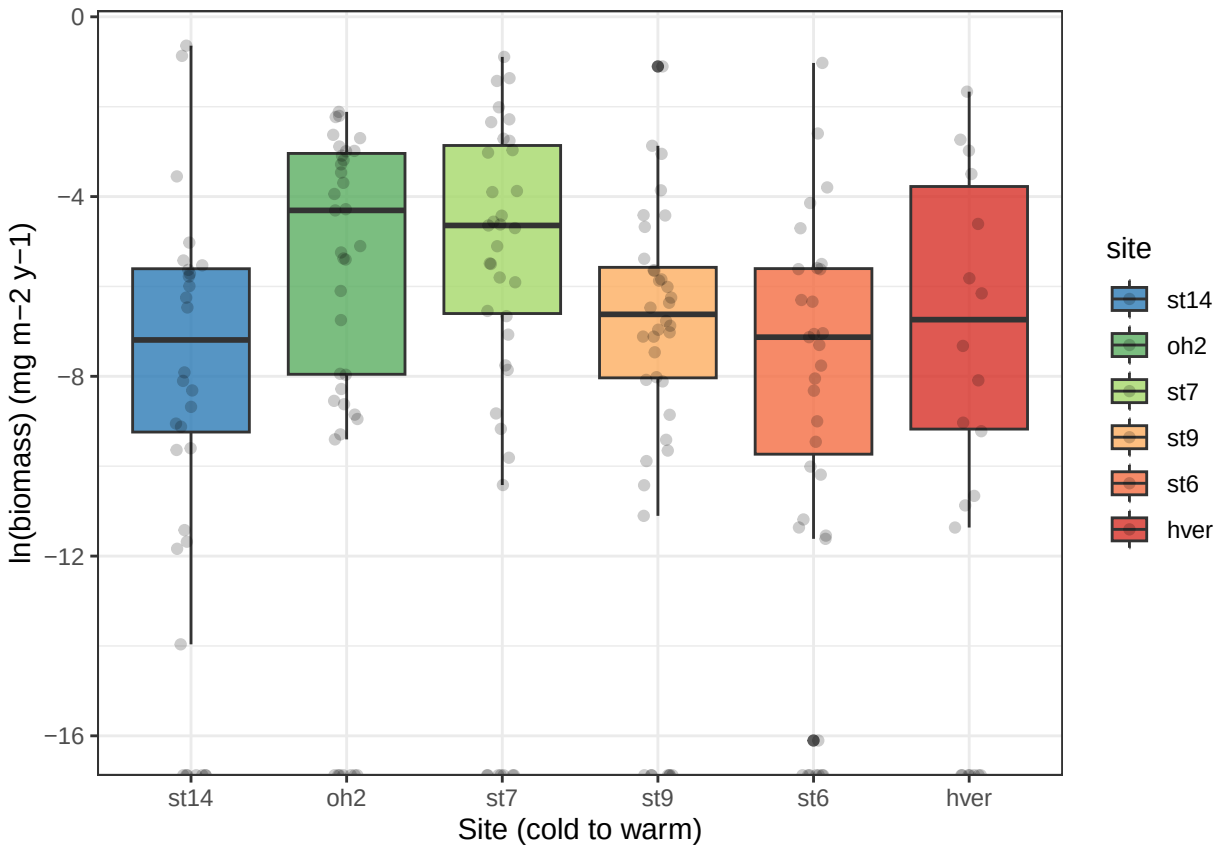
```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```




```r
# Quick check of abundance-biomass relationship
ggplot(community_annual,aes(x=site, y=log(bio_gm2y), fill = site)) +
  geom_boxplot(alpha = 0.8) +
  geom_jitter(alpha = 0.2, width = 0.1) +
  scale_fill_manual(values = site_colors)+
  theme_bw()+
    labs(x = "Site (cold to warm)",
       y = "ln(biomass) (mg m-2 y-1)")
```

```
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```

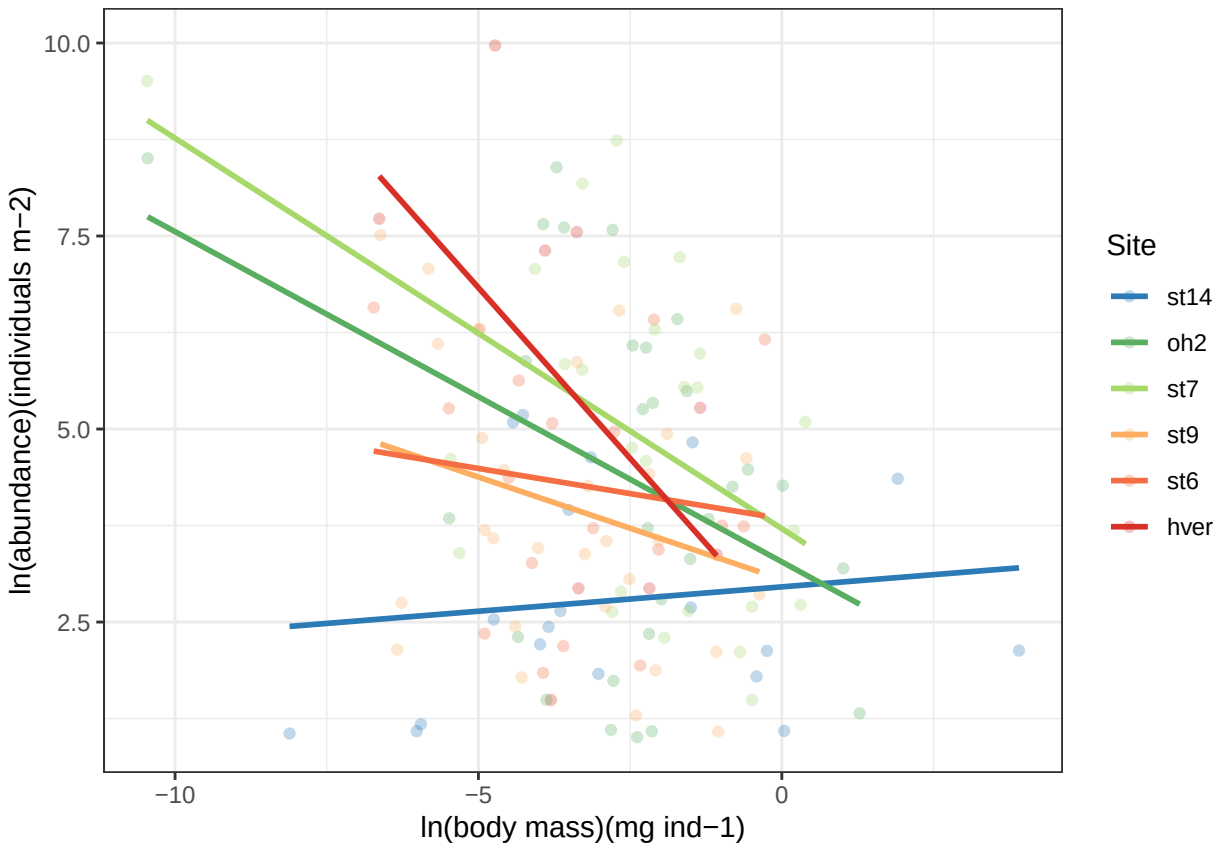



```r
# Plot mass-abundance relationship across thermal gradient
ggplot(community_annual, aes(x = log(g_indy * 1000), y = log(abun_indm2y), group = site, color = site))
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = F) +
  scale_color_manual(values = site_colors)+
  theme_bw() +
    labs(x = "ln(body mass)(mg ind-1)",
         y = "ln(abundance)(individuals m-2)",
         color = "Site") +
  scale_y_continuous(limits = c(1,10))
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 70 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 70 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Results: Abundance~Biomass relationship in not as linear as I thought it should be - this could be from variation in body size contributing to higher biomasses, which could mean I need to consider taking a different approach for summarizing the bootstrap estimates.

The other thing to note is the slope from the M-N scaling (last graph). We might predict more small organisms and fewer large organisms and/or decreasing body size with warming, leading to a steeper MN slope. We see this in the warmest stream (hver), but not necessarily in st6 or st9. Lots of spread with this though...

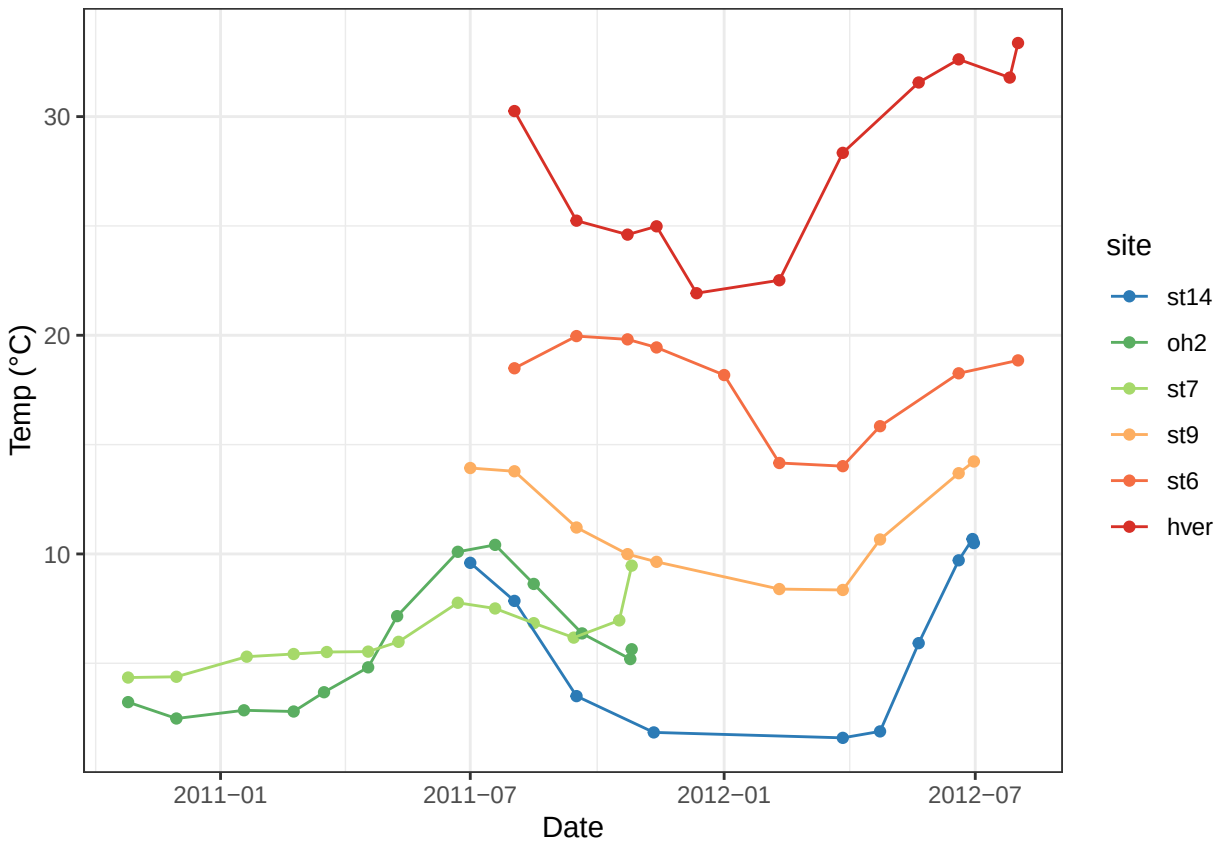
Stream temperatures and Boltzmann-Arrhenius energy kinetics

Water temperatures (annual_tempkt and stream_temps_full) are stored as inverse temperatures ($1/k_BT$). Inverse temperature ($1/k_BT$) is used to account for the Boltzmann-Arrhenius effect on metabolism. It effectively captures the "thermal kinetic energy" available in the ecosystem, which has strong implications on energy flow. For example, hver is the warmest stream in the gradient and has a kT = 38.6 eV^{-1}, whereas st14, the coldest stream, has a kT = 41.7 eV^{-1}. Biologically this means there is less kinetic energy in st14, potentially leading to slower metabolic rates and energy flow through the system.

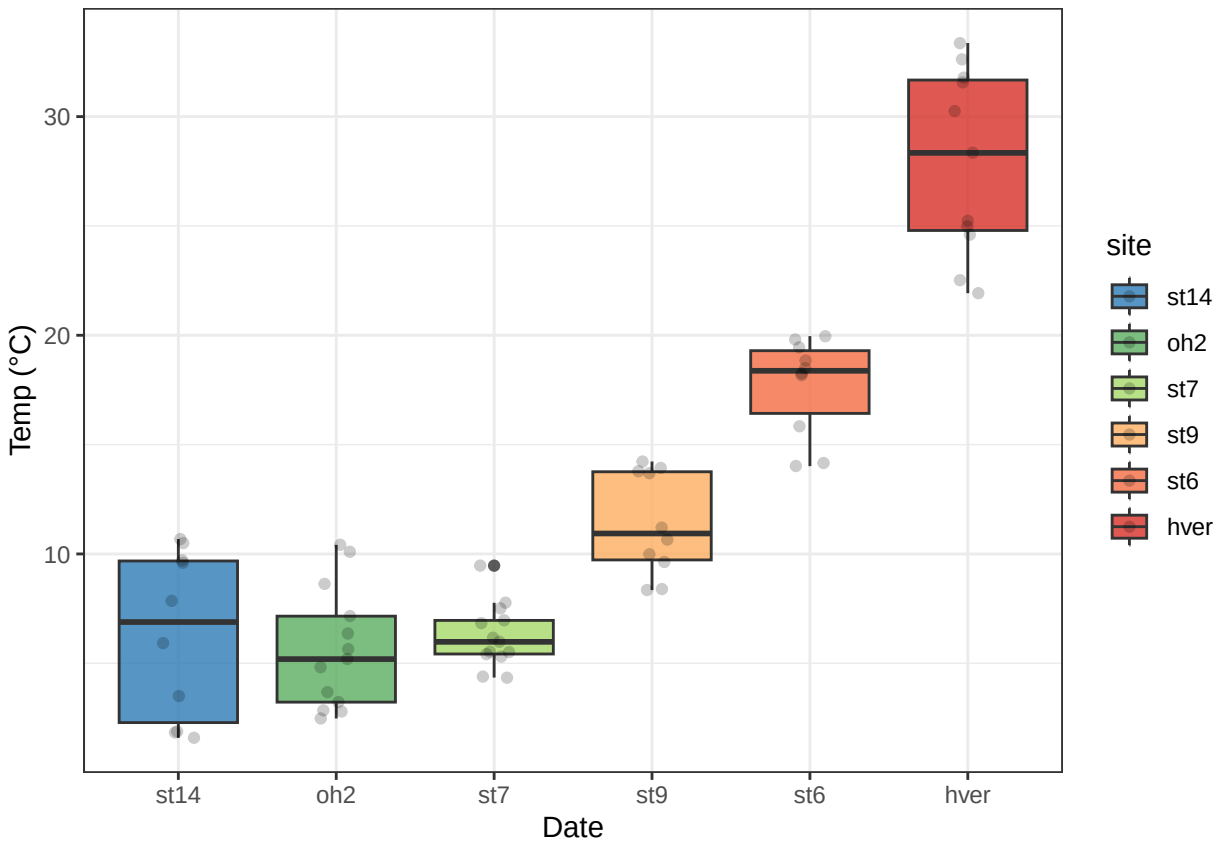
To provide biological realism for plotting, I convert inverse temperature into Kelvin and Celsius at annual and seasonal time points.

```
# Boltzmann constant
k_B <- 8.617e-5

# Pivot dataframe and calculate Kelvin and Celsius
annual_stream_temps <- annual_tempkt %>%
  pivot_longer(cols = everything(),
               names_to = "site",
               values_to = "Kt") %>%
```

```r
  # Clean up the site names (remove the '_tempC' suffix)
  mutate(site = gsub("_tempC", "", site)) %>%
  # Calculate Kelvin and Celsius
  mutate(
    Kelvin = 1 / (k_B * Kt),
    Celsius = Kelvin - 273.15
  )

# Order site by increasing stream temperature
annual_stream_temps <- annual_stream_temps %>%
  mutate(site = recode(site, "Hver" = "hver"), # rename Hver to hver
         site = factor(site,
                       levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
                       ordered = TRUE))

# Do the same for the full stream temperature df, first converting the list to a tibble
stream_temps_full <- bind_rows(stream_temps_full) %>%
  as_tibble()

stream_temps_full <- stream_temps_full %>%
  mutate(kT = tempkt,
         kelvin = 1 / (k_B * kT),
         celcius = kelvin - 273.15,
         site = factor(SITE,
                       levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
                       ordered = TRUE))

# Plot temp through time
ggplot(stream_temps_full, aes(x=DATE,y=celcius,color=site))+
  geom_point() +
  geom_line() +
  scale_color_manual(values = site_colors) +
  labs(x = "Date", y = "Temp (\u00B0C)") +
  theme_bw()
```




```r
# Plot temp through time
ggplot(stream_temps_full, aes(x=site,y=celcius,fill=site))+
  geom_boxplot(alpha = 0.8) +
  geom_jitter(alpha = 0.2, width = 0.1) +
  scale_fill_manual(values = site_colors) +
  labs(x = "Date", y = "Temp (\u00B0C)") +
  theme_bw()
```



Results: Cool to see the temperature gradient and seasonal variability, though seasonal fluctuations are dampened from geothermal influence.

Assimilation efficiency

Assimilation efficiency (AE) represents the proportion of consumed food that an organism can actually use for metabolism and growth. Not all ingested food is useful: some is lost as waste (e.g., undigested material in feces). Additionally, because not all food is nutritionally the same, AE can be customized based on the type of prey eaten. I define course values of AE based on previous literature estimates (Gauzens. et al. 2019) using a for loop, assigning animal AE as 0.906, plant as 0.545, and detritus as 0.158.

```
ae_df <- community_annual %>%
  ungroup() %>%
  # keep unique combinations of site and taxon
  distinct(site, taxon_id) %>%
  # apply ae values to each resource and other taxon
  mutate(ae = case_when(
    taxon_id == "amorphous_detritus" ~ 0.158,
    taxon_id %in% c("plant_material", "green_algae", "filamentous", "diatom", "cyanobacteria") ~ 0.545,
    TRUE ~ 0.906)) %>% # Default for everything else (animals/inverts)
  mutate(site = factor(site,
                       levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
                       ordered = TRUE))
```

This produced a df of assimilation efficiencies for each taxon/resource matching the structure of community_annual. We could tinker with AE later on, as others have found it can change based on temperature (Lang et al 2017 - higher temp; higher AE) or resource stoichiometry (Jochum et al. 2017 - higher N; higher AE).

Metabolic rates and energy demand

Fluxweb assumes energy loss to predation and metabolism. To define metabolic energy loss, we use the metabolic theory of ecology (Brown et al. 2004), which assumes metabolic rates scale with body mass following a power law:

$$B = B_0 M^b$$

where: B is the metabolic rate, B_0 is a normalization constant, M is body mass (mg/ind), and b is the scaling exponent (3/4 scaling is the standard MTE assumption for whole-organism metabolic rates).

Incorporating environmental temperature

As discussed under the temperature section, organisms kinetics increase with temperature. This means that metabolic rate also respond strongly to environmental temperature, especially for ectothermic organisms like fish and invertebrates. To reflect this, we can integrate temperature into metabolic rates following the approach of Schwarz et al. 2017.

$$B = B_0 M^b e^{-\frac{E}{k_B T}}$$

Where:
- B is the temperature-dependent metabolic rate,
- B_0 is a normalization constant,
- M is ind body mass (g),
- b is the allometric exponent (typically 3/4),
- E is the activation energy (eV),
- k_B is the Boltzmann constant (8.617 × 10-5 eV/K),
- T is absolute temperature in Kelvin (K).

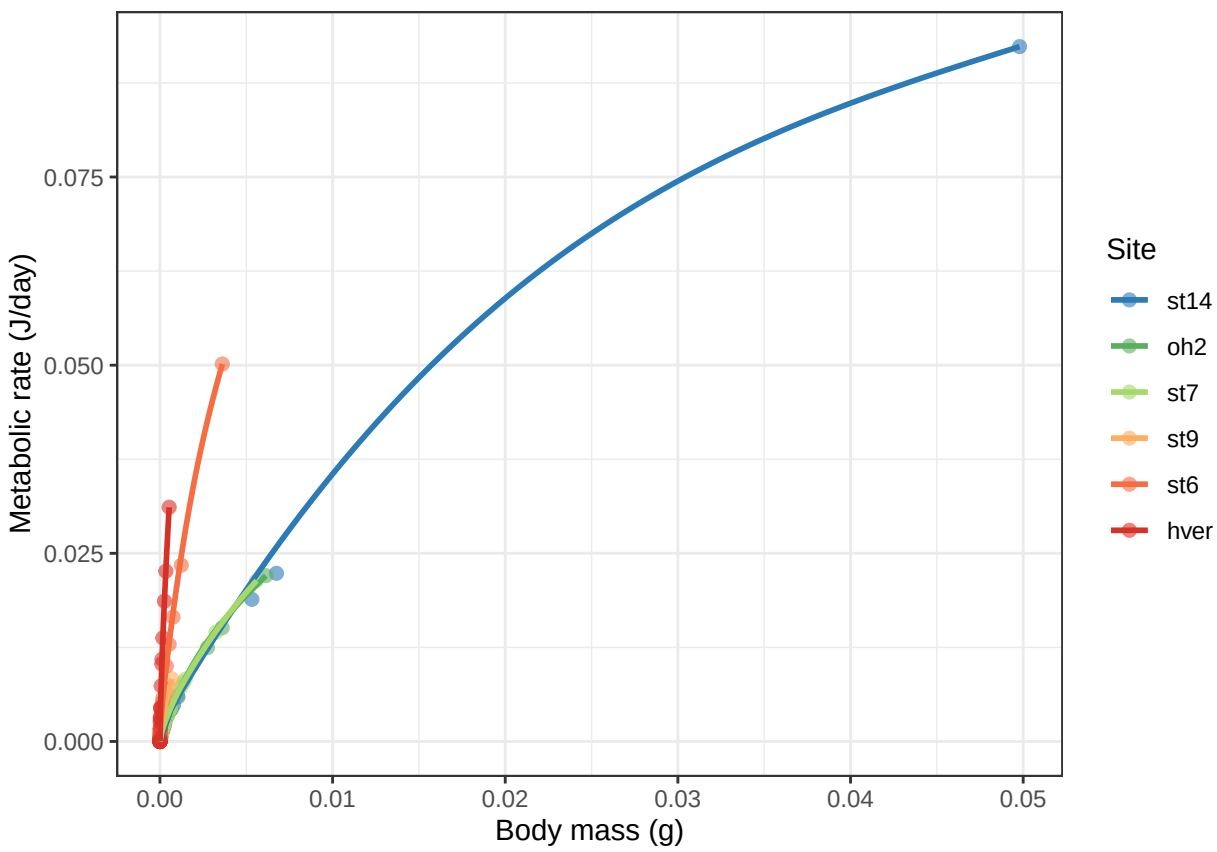
Here, I estimate taxon-specific energy loss to metabolism using the approach of Jochum et al. 2021 and Schwarz et al. 2017. It uses parameter estimates from MTE (Brown et al. 2004) to calculate a per-capita metabolic rate based on body mass (g/ind) and environmental temperature. I use annual averages of body mass and temperature for now - we could explore if/how temporal patterns of body size and temperature influence metabolism later. Importantly, Gauzens et al. 2019 points out that "It is quite common in food webs to have nodes such as 'detritus' or 'dissolved organic matter'. Values for the metabolic rates of such nodes can be set to NA if they are basal and zero in any case." Therefore, I set the basal resources metabolism to 0.

As I discuss after I caluclate these measurements, the units for metabolic rates have been confusing. I believe the method is use calculates a per-capita metabolic rate (rate of energy expenditure per individual) in Watts (Joules/sec).

```
# Create df for energy losses to metabolism with community_annual
losses <- inner_join(community_annual,
                     annual_stream_temps,
                     by = "site") %>%
  group_by(site, taxon_id, Kt, Kelvin, Celsius) %>%
  summarise(bodymass_g = mean(g_indy, na.rm = TRUE), # individual body mass of taxon (g/ind)
            bodymass_mg = mean(g_indy * 1000, na.rm = TRUE), # ind/mg
            .groups = "drop")

# Define parameters for equation
boltz <- 0.00008617343 # Boltzmann constant (eV/Kelvin)
E <- 0.69 # activation energy (eV)
b <- 0.71 # scaling exponent for invertebrates from MTE (dimensionless)
ln_i <- 17.17 # normalization constant (intercept) for invertebrates from MTE (intercept in log space,
```

```r
# Estimate per-capita metabolic rate for each taxon based on individual body mass (g ind-1) and average
losses$metrate_J_per_sec <- exp((b * log(losses$bodymass_g) + ln_i) - (E/(boltz*(losses$Kelvin))))

losses$metrate_J_per_day <- losses$metrate_J_per_sec * 86400 #number of seconds in a day

# Calculate mass-specific loss (J day-1 gram-1)
losses$metrate_J_day_gram <- losses$metrate_J_per_day/losses$bodymass_g

# Replace NA with 0 for basal resources
losses[sapply(losses, is.nan)] <- 0


# Plot metabolic rates as a function of body mass (g)
ggplot(losses, aes(x = bodymass_g, y = metrate_J_per_day, color = site)) +
  geom_point(alpha = 0.6, size = 2) +
  labs(x = "Body mass (g)",
       y = "Metabolic rate (J/day)",
       color = "Site") +
  geom_smooth(se = F)+
  scale_color_manual(values = site_colors) +
  theme_bw()
```

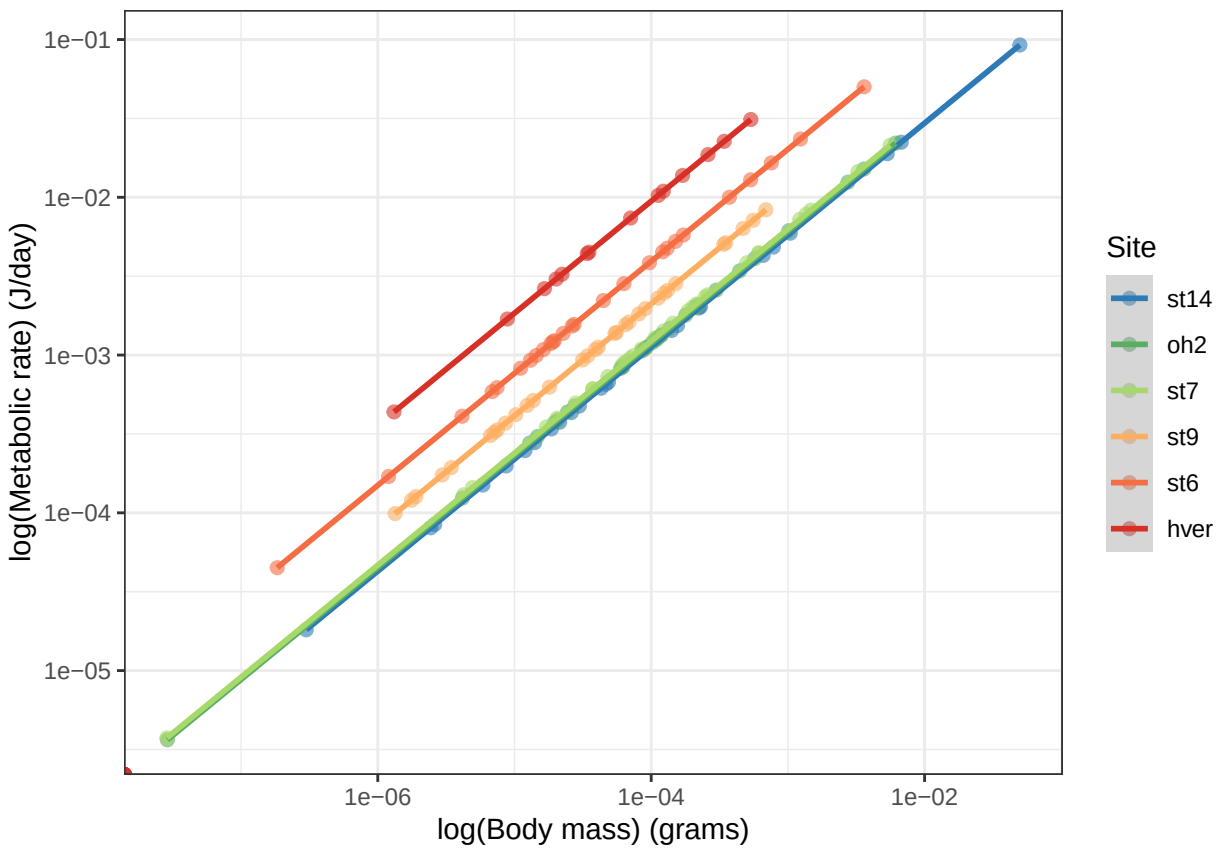
```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```r
# Same plot but in log space with line of best fit
ggplot(losses, aes(x = bodymass_g, y = metrate_J_per_day, color = site)) +
  geom_point(alpha = 0.6, size = 2) +
```

```
  geom_smooth(method = "lm", se = TRUE) +
  scale_x_log10() +
  scale_y_log10() +
  scale_color_manual(values = site_colors) +
  theme_bw() +
  labs(
    x = "log(Body mass) (grams)",
    y = "log(Metabolic rate) (J/day)",
    color = "Site"
  )
```

```
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
## Warning in scale_x_log10(): log-10 transformation introduced infinite values.
## Warning in scale_y_log10(): log-10 transformation introduced infinite values.
## `geom_smooth()` using formula = 'y ~ x'
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

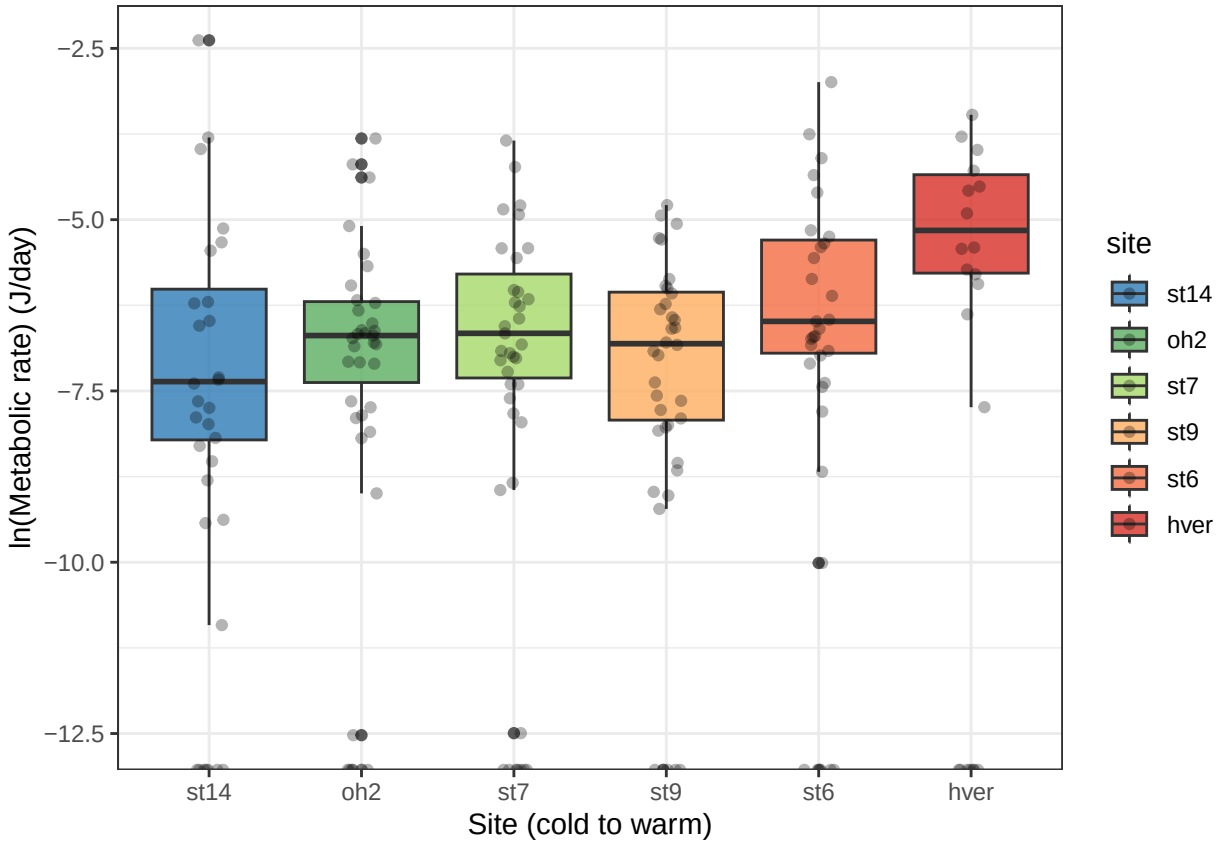



```
# metabolic rates shift with temperature
# also interesting to see the variation in body mass across communities

# Boxplot of metabolic rates across the thermal gradient
ggplot(losses, aes(x = site, y = log(metrate_J_per_day), fill = site)) +
  geom_boxplot(alpha = 0.8) +
```

```r
  geom_jitter(alpha = 0.3, width = 0.1) +
  scale_fill_manual(values = site_colors) +
  theme_bw()+
    labs(
    x = "Site (cold to warm)",
    y = "ln(Metabolic rate) (J/day)",
    color = "Site"
  )
```

```
## Ignoring unknown labels:
## * colour : "Site"
```

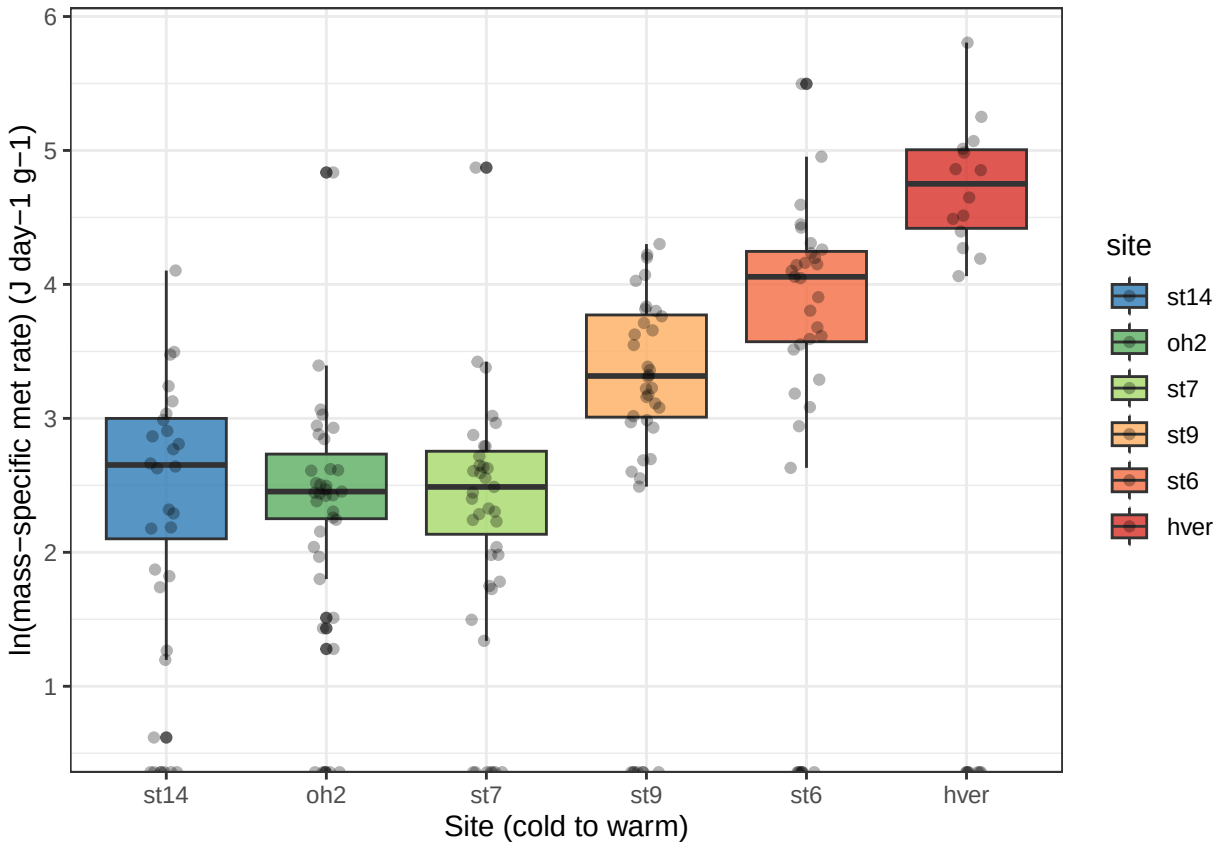
```
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```r
# interesting results, maybe body mass matters more to the metabolic rates than temp

# Mass-corrected metabolic rate across the gradient
ggplot(losses, aes(x = site, y = log(metrate_J_day_gram), fill = site)) +
  geom_boxplot(alpha = 0.8) +
  geom_jitter(alpha = 0.3, width = 0.1) +
  scale_fill_manual(values = site_colors) +
  theme_bw()+
    labs(
    x = "Site (cold to warm)",
    y = "ln(mass-specific met rate) (J day-1 g-1)",
    color = "Site"
  )
```

```
## Ignoring unknown labels:
## * colour : "Site"
```

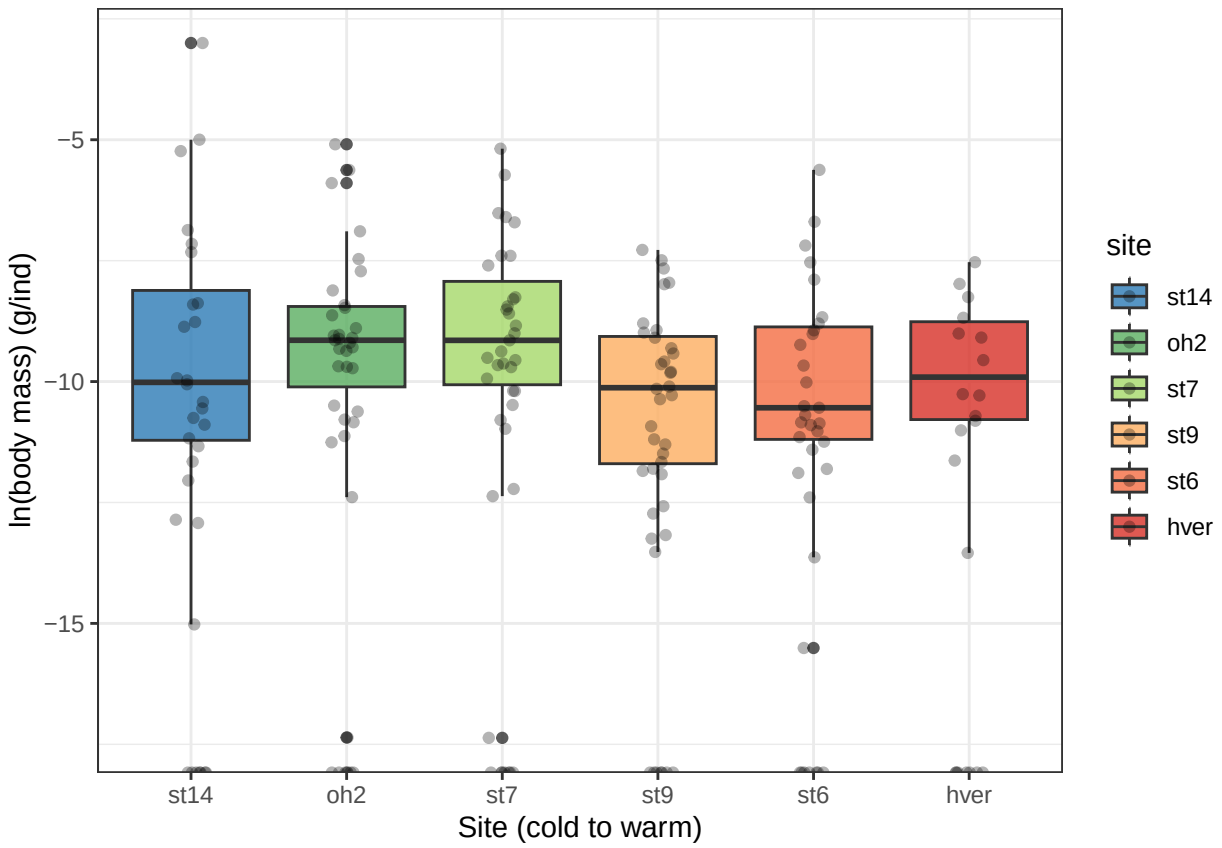
```
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```




```r
# check body mass across the gradient
ggplot(losses, aes(x = site, y = log(bodymass_g), fill = site)) +
  geom_boxplot(alpha = 0.8) +
  geom_jitter(alpha = 0.3, width = 0.1) +
  scale_fill_manual(values = site_colors) +
  theme_bw()+
    labs(
    x = "Site (cold to warm)",
    y = "ln(body mass) (g/ind)",
    color = "Site"
  )
```

```
## Ignoring unknown labels:
## * colour : "Site"
```

```
## Warning: Removed 42 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Results: Losses to metabolism scale with temperature and body mass, but smaller organisms in warm streams keep metabolic demand down.

Using parameters from the MTE, I estimated per-capita metabolic rates (J/sec) for consumers in each stream. The results follow MTE predictions that metabolism scales with body mass (grams per individual) linearly in log space. In absolute terms, this means that energetic demands to metabolism will exponentially increase for larger organisms in warmed streams. Interestingly, however, individual body masses decrease with warming, suggesting that larger bodied organisms have been filtered from these warmed communities. So we see overlap in metabolic rates - temperature increases metabolic rates, but body sizes decrease with warming.

This part is where it could be cool to use some respiration rates from Sam's MSc to empirically derive the parameters to estimate metabolism (across body sizes and temperature of Euke). If *acclimation* occurs across the thermal gradient or body sizes, then then parameters would have less of an effect on metabolic rates, lowering energy demand at the food web level.

Diet matrix

The raw diet matrix is a list containing posterior estimates of diet proportion (see Junker et al.). I take the seasonal (e.g., seasons = "first", "second", "third") and annual average of consumer diets for each stream using a for loop. The result is a sp by sp matrix of the community, with the proportional diet contribution for each consumer. Seasonal dates are "first" is Jan - Apr, "second" is May - August, and "third" is Sept - Dec.

Just like the community data, there are spelling/lumping errors with the taxon_id. I write a function to clean the list based on name_cleaner which was applied to the community structure data. This should fix spelling errors and lumped taxa, taking a mean diet proportion for the later (mean of Naidid 1 and Naidid 2 diet = Naidid diet).

```
# Create a list to store results
mean_diets <- list()
```

```r
# Loop over each site in diet_matrices
for (site_name in names(diet_matrices)) {

  site_data <- diet_matrices[[site_name]]

  # Calculate mean diet for each season
  season_means <- lapply(site_data, function(season_list) {
    season_array <- simplify2array(season_list)
    apply(season_array, c(1, 2), mean)  # mean across iterations
  })

  # Compute annual average (mean across seasons)
  n_seasons <- length(season_means)
  annual_mean <- Reduce(`+`, season_means) / n_seasons

  # Store results for this site
  mean_diets[[site_name]] <- list(
    season_means = season_means,
    annual = annual_mean
  )
}

# Function to clean a single matrix
clean_matrix <- function(mat, cleaner) {
  # Get current names and map to final names
  curr_names <- rownames(mat)

  name_map <- tibble(old_name = curr_names) %>%
    left_join(cleaner, by = "old_name") %>%
    mutate(final_name = coalesce(final_name, old_name))

  new_names <- name_map$final_name

  # Collapse Rows (Consumers)
  # We sum them first, then divide by the number of lumped taxa to get the average diet
  row_counts <- table(new_names)
  mat_collapsed_rows <- rowsum(mat, new_names)
  mat_averaged_rows <- mat_collapsed_rows / as.vector(row_counts[rownames(mat_collapsed_rows)])

  # Collapse Columns (Prey)
  # For prey, we simply sum the proportions
  # (Transpose, collapse 'rows' of the transpose, then transpose back)
  final_mat <- t(rowsum(t(mat_averaged_rows), new_names))

  return(final_mat)
}

# Apply this to your nested list
mean_diets_clean <- map(mean_diets, function(site_list) {
  # Clean the annual matrix
  site_list$annual <- clean_matrix(site_list$annual, name_cleaner)

  # Clean the seasonal matrices (first, second, third)
```

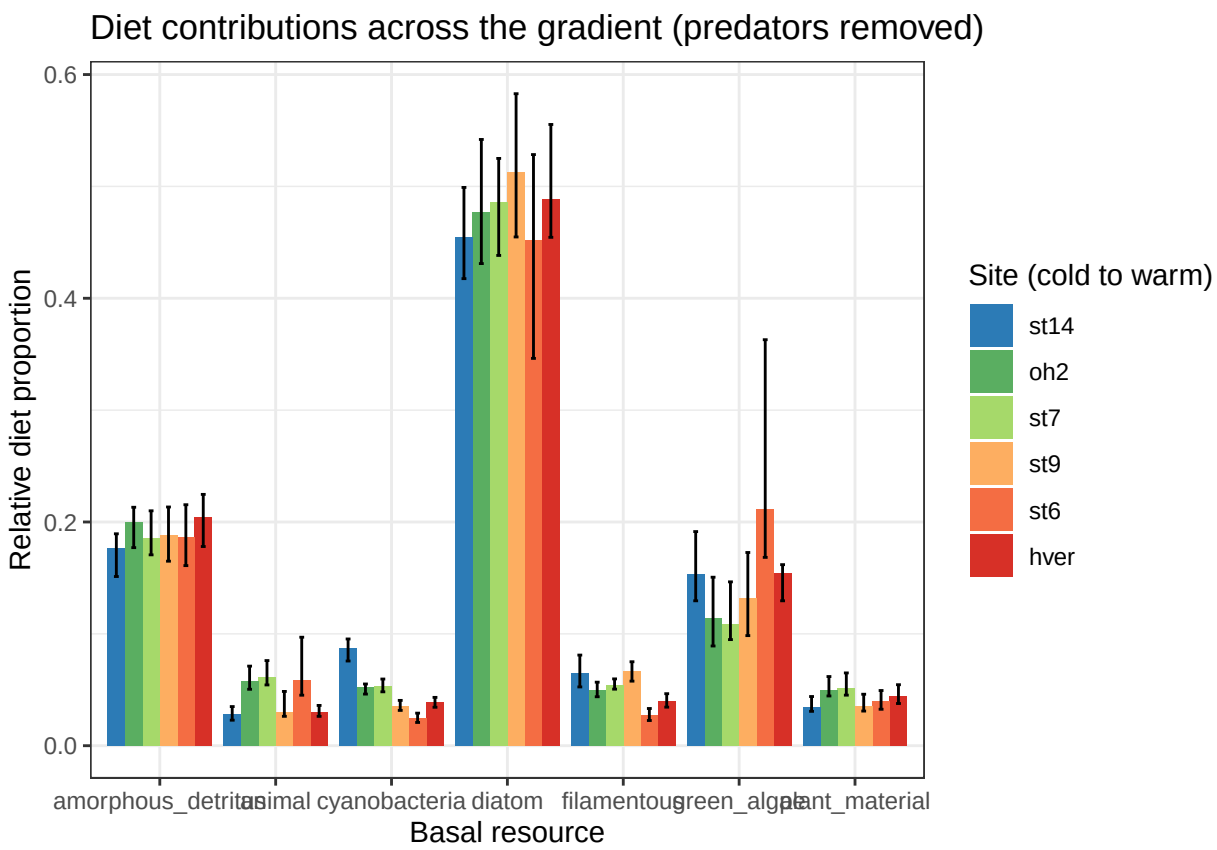
```r
  site_list$season_means <- map(site_list$season_means, ~clean_matrix(.x, name_cleaner))

  return(site_list)
})


# I noticed that st14 did not have diet proportions for Simulium vernum
# I use oh2 (similar temp) Simulium vernum diet proportions and include in st14 matrix

# Extract the "Donor" column from oh2
# We use the annual matrix as the source
donor_col <- mean_diets_clean$oh2$annual[, "Simulium vernum"]

# Update st14
# Check if the species already exists in st14; if not, we add it
if (!"Simulium vernum" %in% colnames(mean_diets_clean$st14$annual)) {

  # Get current dimensions
  old_mat <- mean_diets_clean$st14$annual
  new_names <- c(colnames(old_mat), "Simulium vernum")

  # Create a new expanded matrix
  new_mat <- matrix(0, nrow = length(new_names), ncol = length(new_names),
                    dimnames = list(new_names, new_names))

  # Fill in the old data
  new_mat[rownames(old_mat), colnames(old_mat)] <- old_mat

  # Insert the donor column for the new species
  common_prey <- intersect(names(donor_col), colnames(new_mat))
  new_mat["Simulium vernum", ] <- 0 # Row is 0 (it is not eaten)
  new_mat[common_prey, "Simulium vernum"] <- donor_col[common_prey]

  # Save it back
  mean_diets_clean$st14$annual <- new_mat
} else {
  # If the species existed but the column was just empty (zeros)
  mean_diets_clean$st14$annual[, "Simulium vernum"] <- donor_col[rownames(mean_diets_clean$st14$annual)]
}
```

Visualize relative proportion of diet to consumers across the thermal gradient

```r
# Summarize the list to look at the proportions of consumed food across consumers and the gradient, rem
basal_importance <- lapply(mean_diets, function(x) {

  annual_mat <- x$annual
  # only summarize basal resources (last 6 rows in the matrix)
  basal_mat <- annual_mat[(nrow(annual_mat) - 6):nrow(annual_mat), , drop = FALSE]

  data.frame(
    resource = rownames(basal_mat),
    # estimate minimum contribution of resource to consumers
    min_importance = apply(
```

```r
      basal_mat, 1,
      function(v) min(v[v > 0], na.rm = TRUE)
    ),
    # estimate mean contribution of resource to consumers
    mean_importance = mapply(
      function(v, res) {
        # For animal, exclude 1s before computing mean (excludes the influence of predators)
        if (res == "animal" && any(v == 1, na.rm = TRUE)) {
          v2 <- v[v < 1 & v > 0]
          if (length(v2) == 0) NA else mean(v2, na.rm = TRUE)
        } else {
          mean(v[v > 0], na.rm = TRUE)
        }
      },
      split(basal_mat, row(basal_mat)),
      rownames(basal_mat)
    ),
    # estimate max contribution of resource to consumers
    max_importance = mapply(
      function(v, res) {
        if (res == "animal" && any(v == 1, na.rm = TRUE)) {
          v2 <- v[v < 1 & v > 0]
          if (length(v2) == 0) 1 else max(v2, na.rm = TRUE)
        } else {
          max(v, na.rm = TRUE)
        }
      },
      split(basal_mat, row(basal_mat)),
      rownames(basal_mat)
    )
  )
})


# Convert the list to a dataframe for plotting
basal_importance_df <- do.call(
  rbind,
  lapply(names(basal_importance), function(site) {
    df <- basal_importance[[site]]
    df$site <- site
    df
  })
)

# Make site ordered by temperature
basal_importance_df <- basal_importance_df %>%
  mutate(
    site = factor(
      site,
      levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
      ordered = TRUE
    )
  )
```

```r
# Resource contributions to all consumers across the gradient, error bars are the min and max
ggplot(basal_importance_df, aes(x = resource, y = mean_importance, fill = site)) +
  geom_col(position = position_dodge(width = 0.9)) +
  geom_errorbar(
    aes(ymin = min_importance, ymax = max_importance),
    position = position_dodge(width = 0.9),
    width = 0.25) +
  scale_fill_manual(values = site_colors) +
  labs(
    y = "Relative diet proportion",
    x = "Basal resource",
    fill = "Site (cold to warm)",
    title = "Diet contributions across the gradient (predators removed)") +
  theme_bw()
```



Results: Diatoms dominate, and the relative proportion of basal resources does not vary across the thermal gradient

When averaged across consumers at each site (removing predators), diatoms are the dominant food resource, and generally, basal resource contributions to consumer diets does not vary across the gradient. This means that **major differences in energy fluxes** will likely stem from community structure, specifically how taxon biomasses and their metabolic rates differ across the gradient.

Combining data to one nested list

Here, I combine the data to one nested list. Each stream (hver,oh2,st14,st6,st7,st9) is its own list, with each data to estimate energy fluxes (biomass (mg/m2), diet matrix, energy losses, and assimilation efficiency [prey

defined]) stored as a vector.

```
# Explore fluxweb vignette to see how data is stored
#species.level
#str(species.level)

# Combine species id, biomass, metabolic rate, and ae into one df
flux_df <- community_annual %>%
  left_join(ae_df, by = c("site", "taxon_id")) %>%
  left_join(losses, by = c("site", "taxon_id")) %>%
  select(site,
         sampling_period,
         taxon_id,
         bio_gm2y,
         ae,
         g_indy,
         metrate_J_per_day,
         metrate_J_day_gram)

flux_all <- flux_df %>%
  ungroup() %>%
  # Group by site to prepare for nesting
  group_by(site) %>%
  # Nest the data into site-specific df
  nest() %>%
  # Transform each df into the list structure that follows Gauzens et al. 2019
  mutate(data = map(data, ~ list(biomasses = .x$bio_gm2y,
                                 efficiencies = .x$ae,
                                 losses = .x$metrate_J_day_gram,
                                 bodymasses = .x$g_indy,
                                 names = .x$taxon_id)
                    )
         ) %>%
  # Turn the two-column tibble (site, data) into a named list
  deframe()

flux_all <- imap(flux_all, function(site_data, site_name) {
  # Grab the specific matrix for this site and add it to the list
  site_data$mat <- mean_diets_clean[[site_name]]$annual
  return(site_data)
})
```

We need to make sure the community structure data aligns with the matrix.

```
# We will iterate through each site in your list
flux_all_aligned <- map(flux_all, function(site_list) {

  # Pull the master name order for this specific site
  target_order <- site_list$names

  # Get the current matrix
  current_mat <- site_list$mat

  # Identify the current order of the matrix names
  current_order <- rownames(current_mat)
```

```r
  # Create a reindexing vector
  # This finds where each 'target' name lives in the 'current' matrix
  reindex_vector <- match(target_order, current_order)

  # Reshuffle the matrix (Rows first, then Columns)
  # This also handles cases where a name exists in 'names' but not 'mat'
  # by creating NA rows/cols, which we then turn to 0
  aligned_mat <- current_mat[reindex_vector, reindex_vector]

  # Safety: Replace any NAs with 0
  # (occurs if a species is in 'names' but was missing from the diet matrix)
  aligned_mat[is.na(aligned_mat)] <- 0

  # Re-assign the correct names to the matrix dimensions
  rownames(aligned_mat) <- target_order
  colnames(aligned_mat) <- target_order

  # Replace the old matrix with the aligned one
  site_list$mat <- aligned_mat

  return(site_list)
})
```

Fluxing

Here I finally calculate energy flux (J m-2 d-1) for each stream. I annotate the units for hver fluxes, and then summarize the results in a dataframe.

```r
# Calculate fluxes in hver
hver_flux <- fluxing(flux_all_aligned[["hver"]][["mat"]], # trophic interactions (proportions; dimensio
                     flux_all_aligned[["hver"]][["biomasses"]], # species biomasses (g m-2)
                     flux_all_aligned[["hver"]][["losses"]], # metabolic loss (J day-1 gram-1)
                     flux_all_aligned[["hver"]][["efficiencies"]], # AE values (dimensionless)
                     bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                     bioms.losses = TRUE, # scales metabolic loss to population biomass
                     ef.level = "prey") # AE depends on prey type (basal resource)

# Total energy (J m-2 d-1) going each taxon
colSums(hver_flux)
```

```
## Cricotopus sylvestris           Hirudinidae    Limnophora riparia
##          17.598702555           0.002494148           0.214063066
##           Lumbricidae          Midge indet.                Naidid
##           0.014741544           0.003531847           9.519430087
##             Nais spp.              Nematoda              Oribatid
##          75.709098818           0.007830553           2.070700484
##             Ostracoda        Radix balthica     Simulium vittatum
##           1.382282073           7.654965489           0.180074678
##  Sperchon glandulosus          Thysanoptera    amorphous_detritus
##           0.051917945           0.030335820           0.000000000
##                animal         cyanobacteria                diatom
##           0.000000000           0.000000000           0.000000000
##           filamentous           green_algae        plant_material
```

```
##           0.000000000           0.000000000           0.000000000
```

```r
# Total energy demand in hver (J m-2 d-1)
sum(hver_flux)
```

```
## [1] 114.4402
```

```r
# Summarize into data.frame
# pivot matrix longer
hver_flux_long <- hver_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")

# CONSUMER SUMMARY
hver_consumers <- hver_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "hver")

site_total <- sum(hver_consumers$energy_flux)

hver_consumers <- hver_consumers %>%
  mutate(
    site_total_flux = site_total,
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- hver_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(hver_flux_long$flux)
  )
```

I repeat this for each stream across the temp gradient.

```r
# Calculate fluxes in st6
st6_flux <- fluxing(flux_all_aligned[["st6"]][["mat"]], # trophic interactions (proportions; dimensionl
                    flux_all_aligned[["st6"]][["biomasses"]], # species biomasses (g m-2)
                    flux_all_aligned[["st6"]][["losses"]], # metabolic loss (J day-1 gram-1)
                    flux_all_aligned[["st6"]][["efficiencies"]], # AE values (dimensionless)
                    bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                    bioms.losses = TRUE, # scales metabolic loss to population biomass
                    ef.level = "prey") # AE depends on prey type (basal resource)

# Summarize into data.frame
# pivot matrix longer
st6_flux_long <- st6_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")
```

```r
# CONSUMER SUMMARY
st6_consumers <- st6_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "st6")

site_total <- sum(st6_consumers$energy_flux)

st6_consumers <- st6_consumers %>%
  mutate(
    site_total_flux = site_total,
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- st6_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(st6_flux_long$flux)
  )

# Calculate fluxes in st9
st9_flux <- fluxing(flux_all_aligned[["st9"]][["mat"]], # trophic interactions (proportions; dimensionl
                    flux_all_aligned[["st9"]][["biomasses"]], # species biomasses (g m-2)
                    flux_all_aligned[["st9"]][["losses"]], # metabolic loss (J day-1 gram-1)
                    flux_all_aligned[["st9"]][["efficiencies"]], # AE values (dimensionless)
                    bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                    bioms.losses = TRUE, # scales metabolic loss to population biomass
                    ef.level = "prey") # AE depends on prey type (basal resource)

# Summarize into data.frame
# pivot matrix longer
st9_flux_long <- st9_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")

# CONSUMER SUMMARY
st9_consumers <- st9_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "st9")

site_total <- sum(st9_consumers$energy_flux)

st9_consumers <- st9_consumers %>%
  mutate(
    site_total_flux = site_total,
```

```r
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- st9_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(st9_flux_long$flux)
  )


# Calculate fluxes in st7
st7_flux <- fluxing(flux_all_aligned[["st7"]][["mat"]], # trophic interactions (proportions; dimensionl
                    flux_all_aligned[["st7"]][["biomasses"]], # species biomasses (g m-2)
                    flux_all_aligned[["st7"]][["losses"]], # metabolic loss (J day-1 gram-1)
                    flux_all_aligned[["st7"]][["efficiencies"]], # AE values (dimensionless)
                    bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                    bioms.losses = TRUE, # scales metabolic loss to population biomass
                    ef.level = "prey") # AE depends on prey type (basal resource)

# Summarize into data.frame
# pivot matrix longer
st7_flux_long <- st7_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")

# CONSUMER SUMMARY
st7_consumers <- st7_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "st7")

site_total <- sum(st7_consumers$energy_flux)

st7_consumers <- st7_consumers %>%
  mutate(
    site_total_flux = site_total,
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- st7_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(st7_flux_long$flux)
```

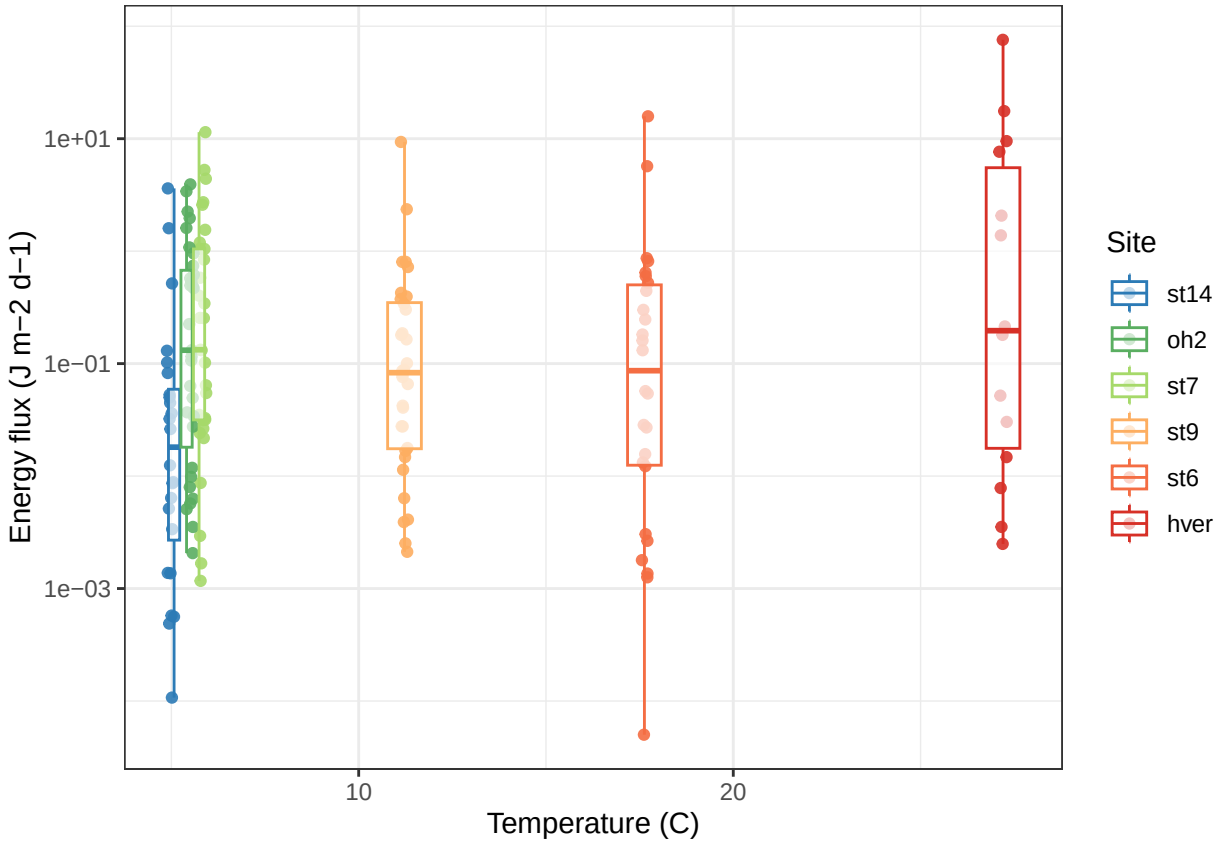
```r
  )


# Calculate fluxes in oh2
oh2_flux <- fluxing(flux_all_aligned[["oh2"]][["mat"]], # trophic interactions (proportions; dimensionl
                    flux_all_aligned[["oh2"]][["biomasses"]], # species biomasses (g m-2)
                    flux_all_aligned[["oh2"]][["losses"]], # metabolic loss (J day-1 gram-1)
                    flux_all_aligned[["oh2"]][["efficiencies"]], # AE values (dimensionless)
                    bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                    bioms.losses = TRUE, # scales metabolic loss to population biomass
                    ef.level = "prey") # AE depends on prey type (basal resource)

# Summarize into data.frame
# pivot matrix longer
oh2_flux_long <- oh2_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")

# CONSUMER SUMMARY
oh2_consumers <- oh2_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "oh2")

site_total <- sum(oh2_consumers$energy_flux)

oh2_consumers <- oh2_consumers %>%
  mutate(
    site_total_flux = site_total,
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- oh2_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(oh2_flux_long$flux)
  )



# Calculate fluxes in st14
st14_flux <- fluxing(flux_all_aligned[["st14"]][["mat"]], # trophic interactions (proportions; dimensio
                    flux_all_aligned[["st14"]][["biomasses"]], # species biomasses (g m-2)
                    flux_all_aligned[["st14"]][["losses"]], # metabolic loss (J day-1 gram-1)
                    flux_all_aligned[["st14"]][["efficiencies"]], # AE values (dimensionless)
                    bioms.prefs = FALSE, # set FALSE because we do not have basal resource abundance
                    bioms.losses = TRUE, # scales metabolic loss to population biomass
```

```r
                      ef.level = "prey") # AE depends on prey type (basal resource)

# Summarize into data.frame
# pivot matrix longer
st14_flux_long <- st14_flux %>%
  as.data.frame() %>%
  rownames_to_column("prey") %>%
  pivot_longer(-prey, names_to = "taxon_id", values_to = "flux")

# CONSUMER SUMMARY
st14_consumers <- st14_flux_long %>%
  group_by(taxon_id) %>%
  summarise(energy_flux = sum(flux), # units are J m-2 d-1
            .groups = "drop") %>%
  mutate(site = "st14")

site_total <- sum(st14_consumers$energy_flux)

st14_consumers <- st14_consumers %>%
  mutate(
    site_total_flux = site_total,
    pct_flux_total = 100 * energy_flux / site_total_flux
  )

# BASAL SUPPLY
basal_supply <- st14_flux_long %>%
  filter(prey %in% basal_taxa) %>%
  group_by(prey) %>%
  summarise(
    basal_flux = sum(flux), # units are J m-2 d-1
    pct_of_total_basal = 100 * basal_flux / sum(basal_flux),
    pct_of_total_web = 100 * basal_flux / sum(st14_flux_long$flux)
  )
```

```r
consumer_fluxes <- bind_rows(
  hver_consumers,
  st6_consumers,
  st9_consumers,
  st7_consumers,
  oh2_consumers,
  st14_consumers)

# Reorder rows, add annual temp in Celcius for each site, and remove basal taxa
consumer_fluxes <- consumer_fluxes %>%
  select(site, taxon_id, energy_flux, site_total_flux, pct_flux_total) %>%
  left_join(annual_stream_temps %>% select(site, Celsius), by = "site") %>%
  rename(annual_temp_C = Celsius) %>%
  filter(!taxon_id %in% basal_taxa,
         !energy_flux == 0) %>%
  mutate(site = factor(site,
                       levels = c("st14", "oh2", "st7", "st9", "st6", "hver"),
                       ordered = TRUE))
```

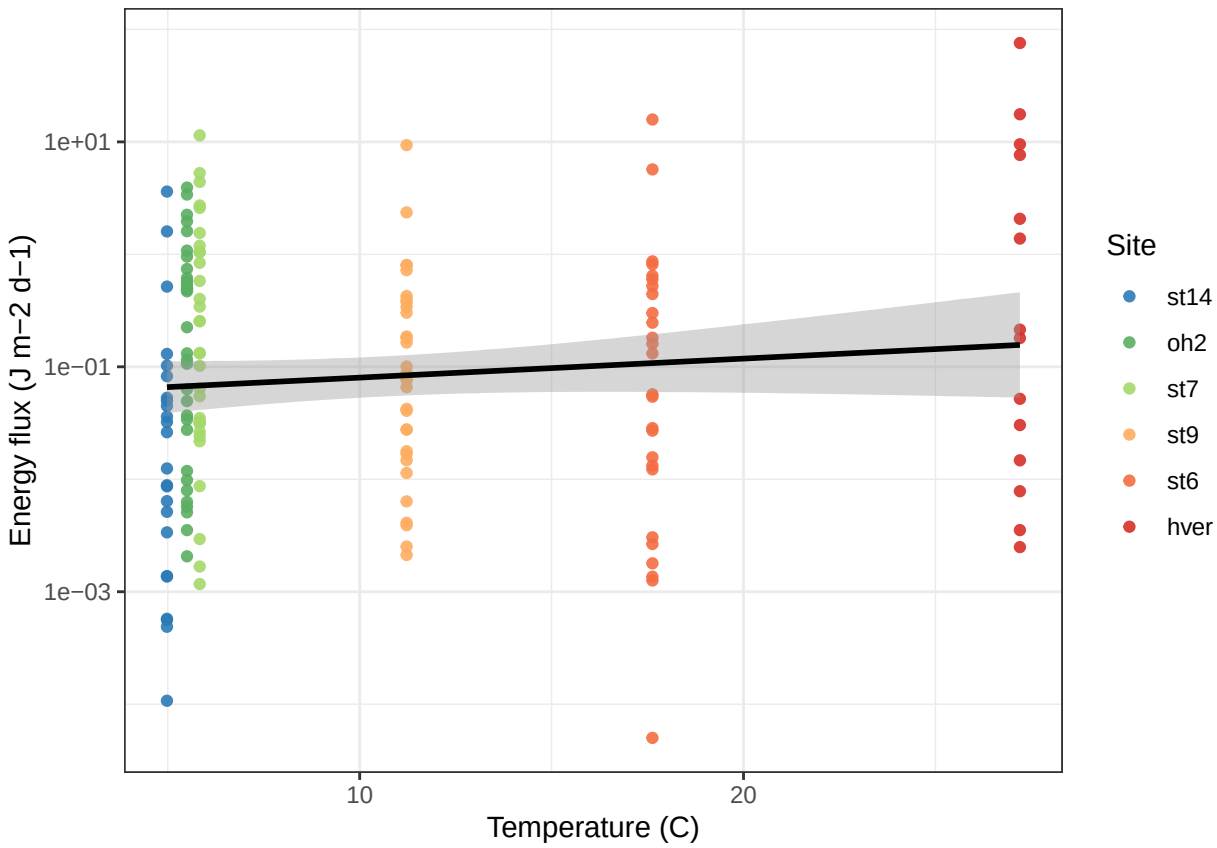
Visualize fluxes

```
ggplot(consumer_fluxes, aes(x=annual_temp_C, y = energy_flux, color = site)) +
  geom_jitter(alpha = 0.9, width = 0.1) + # each dot represents a consumer
  geom_boxplot(alpha = 0.7, width = 1) + # distribution of fluxes
  scale_color_manual(values = site_colors) +
  scale_y_log10() +
  theme_bw()+
    labs(
    x = "Temperature (C)",
    y = "Energy flux (J m-2 d-1)",
    color = "Site"
  )
```

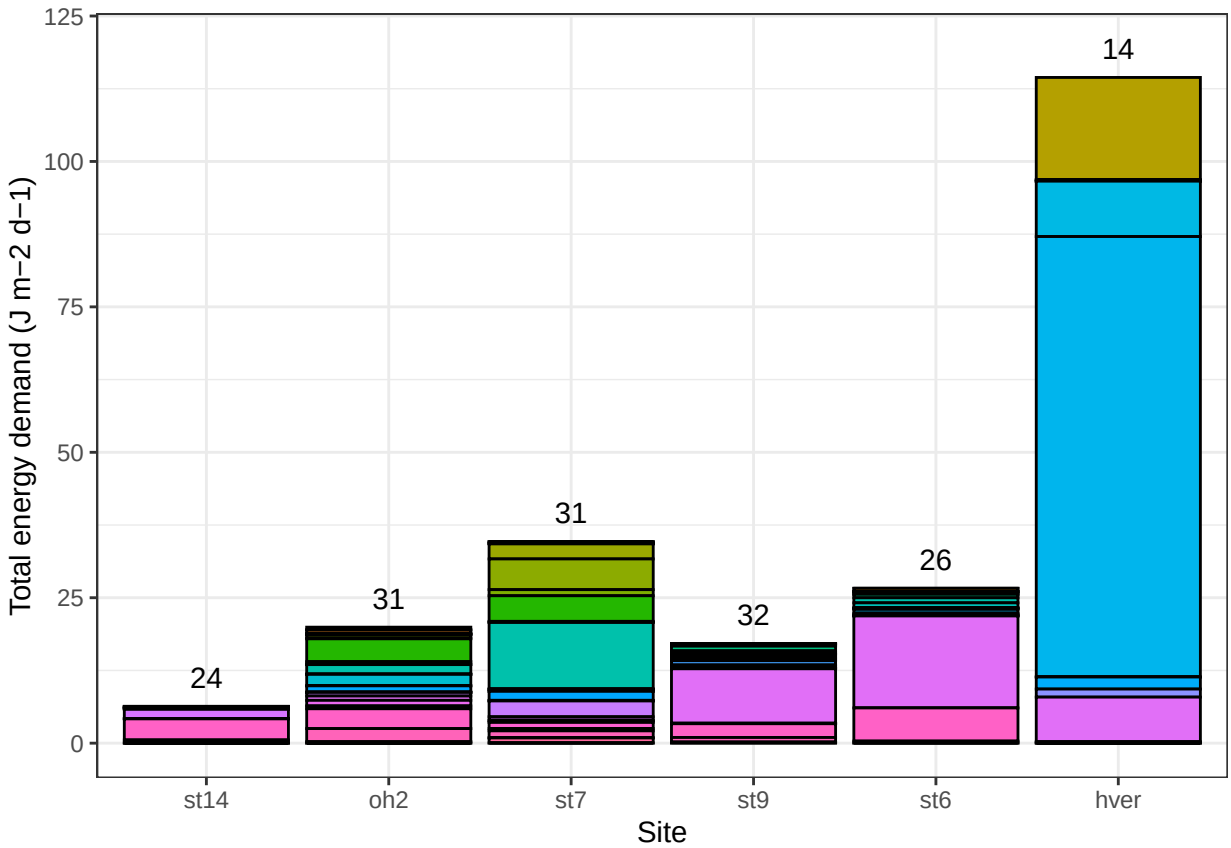



```
ggplot(consumer_fluxes, aes(x=annual_temp_C, y = energy_flux, color = site)) +
  geom_point(alpha = 0.9) + # each dot represents a consumer
  geom_smooth(aes(group = 1), method = "lm", color = "black") +
  scale_color_manual(values = site_colors) +
  scale_y_log10() +
  theme_bw()+
    labs(
    x = "Temperature (C)",
    y = "Energy flux (J m-2 d-1)",
    color = "Site"
  )
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
# Calculate number of consumers per site
consumer_counts <- consumer_fluxes %>%
  group_by(site) %>%
  summarise(n_consumers = n(),
            site_total_flux = first(site_total_flux),
            .groups = "drop")

# Plot total energy demand across the gradient with consumer total above column
ggplot(consumer_fluxes, aes(x = site, y = energy_flux, fill = taxon_id)) +
  geom_col(color = "black") +
  geom_text(
    data = consumer_counts,
    aes(x = site, y = site_total_flux + 5, label = n_consumers),
    inherit.aes = FALSE
  ) +
  ylab("Total energy demand (J m-2 d-1)") +
  xlab("Site") +
  theme_bw() +
  theme(legend.position = "none")
```



Stability

Convert the flux matrix to stability. There's two metrix in fluxweb, *stability.value* and *make.stable*. *Stability.value*

```
hver_flux
```

```
##                        Cricotopus sylvestris  Hirudinidae Limnophora riparia
## Cricotopus sylvestris             0.0000000 0.000000e+00          0.0000000
## Hirudinidae                       0.0000000 0.000000e+00          0.0000000
## Limnophora riparia                0.0000000 0.000000e+00          0.0000000
## Lumbricidae                       0.0000000 0.000000e+00          0.0000000
## Midge indet.                      0.0000000 0.000000e+00          0.0000000
## Naidid                            0.0000000 0.000000e+00          0.0000000
## Nais spp.                         0.0000000 0.000000e+00          0.0000000
## Nematoda                          0.0000000 0.000000e+00          0.0000000
## Oribatid                          0.0000000 0.000000e+00          0.0000000
## Ostracoda                         0.0000000 0.000000e+00          0.0000000
## Radix balthica                    0.0000000 0.000000e+00          0.0000000
## Simulium vittatum                 0.0000000 0.000000e+00          0.0000000
## Sperchon glandulosus              0.0000000 0.000000e+00          0.0000000
## Thysanoptera                      0.0000000 0.000000e+00          0.0000000
## amorphous_detritus                3.5936876 4.441506e-04          0.0000000
## animal                            0.4977173 6.774527e-05          0.2140631
## cyanobacteria                     0.6913273 8.575014e-05          0.0000000
## diatom                            8.7549345 1.385173e-03          0.0000000
## filamentous                       0.6438184 8.603459e-05          0.0000000
```

```
## green_algae                      2.6667639 3.231585e-04          0.0000000
## plant_material                   0.7504535 1.021357e-04          0.0000000
##                        Lumbricidae Midge indet.    Naidid Nais spp.
## Cricotopus sylvestris 0.0000000000 0.0000000000 0.0000000  0.000000
## Hirudinidae           0.0000000000 0.0000000000 0.0000000  0.000000
## Limnophora riparia    0.0000000000 0.0000000000 0.0000000  0.000000
## Lumbricidae           0.0000000000 0.0000000000 0.0000000  0.000000
## Midge indet.          0.0000000000 0.0000000000 0.0000000  0.000000
## Naidid                0.0000000000 0.0000000000 0.0000000  0.000000
## Nais spp.             0.0000000000 0.0000000000 0.0000000  0.000000
## Nematoda              0.0000000000 0.0000000000 0.0000000  0.000000
## Oribatid              0.0000000000 0.0000000000 0.0000000  0.000000
## Ostracoda             0.0000000000 0.0000000000 0.0000000  0.000000
## Radix balthica        0.0000000000 0.0000000000 0.0000000  0.000000
## Simulium vittatum     0.0000000000 0.0000000000 0.0000000  0.000000
## Sperchon glandulosus  0.0000000000 0.0000000000 0.0000000  0.000000
## Thysanoptera          0.0000000000 0.0000000000 0.0000000  0.000000
## amorphous_detritus    0.0033119781 0.0007104420 1.9179362 15.483735
## animal                0.0005307516 0.0001089150 0.2804604  2.234955
## cyanobacteria         0.0006370649 0.0001360267 0.3676402  2.917614
## diatom                0.0066995973 0.0017336404 4.6433594 36.580099
## filamentous           0.0005444359 0.0001348059 0.3703126  3.027512
## green_algae           0.0022139426 0.0005465399 1.5292001 12.121179
## plant_material        0.0008037740 0.0001614774 0.4105212  3.344006
##                           Nematoda   Oribatid  Ostracoda Radix balthica
## Cricotopus sylvestris 0.0000000000 0.00000000 0.00000000      0.0000000
## Hirudinidae           0.0000000000 0.00000000 0.00000000      0.0000000
## Limnophora riparia    0.0000000000 0.00000000 0.00000000      0.0000000
## Lumbricidae           0.0000000000 0.00000000 0.00000000      0.0000000
## Midge indet.          0.0000000000 0.00000000 0.00000000      0.0000000
## Naidid                0.0000000000 0.00000000 0.00000000      0.0000000
## Nais spp.             0.0000000000 0.00000000 0.00000000      0.0000000
## Nematoda              0.0000000000 0.00000000 0.00000000      0.0000000
## Oribatid              0.0000000000 0.00000000 0.00000000      0.0000000
## Ostracoda             0.0000000000 0.00000000 0.00000000      0.0000000
## Radix balthica        0.0000000000 0.00000000 0.00000000      0.0000000
## Simulium vittatum     0.0000000000 0.00000000 0.00000000      0.0000000
## Sperchon glandulosus  0.0000000000 0.00000000 0.00000000      0.0000000
## Thysanoptera          0.0000000000 0.00000000 0.00000000      0.0000000
## amorphous_detritus    0.0015977073 0.42494367 0.28234676      1.5606291
## animal                0.0002326561 0.06287346 0.04165525      0.2010639
## cyanobacteria         0.0003063501 0.08260961 0.05418416      0.2802110
## diatom                0.0037760106 1.00104320 0.67382632      3.7766378
## filamentous           0.0003147040 0.08213175 0.05396697      0.3236378
## green_algae           0.0012675453 0.32714136 0.21665589      1.2239685
## plant_material        0.0003355801 0.08995744 0.05964671      0.2888174
##                       Simulium vittatum Sperchon glandulosus Thysanoptera
## Cricotopus sylvestris       0.000000000          0.000000000 0.0000000000
## Hirudinidae                 0.000000000          0.000000000 0.0000000000
## Limnophora riparia          0.000000000          0.000000000 0.0000000000
## Lumbricidae                 0.000000000          0.000000000 0.0000000000
## Midge indet.                0.000000000          0.000000000 0.0000000000
## Naidid                      0.000000000          0.000000000 0.0000000000
## Nais spp.                   0.000000000          0.000000000 0.0000000000
```

```
## Nematoda                    0.000000000          0.000000000 0.0000000000
## Oribatid                    0.000000000          0.000000000 0.0000000000
## Ostracoda                   0.000000000          0.000000000 0.0000000000
## Radix balthica              0.000000000          0.000000000 0.0000000000
## Simulium vittatum           0.000000000          0.000000000 0.0000000000
## Sperchon glandulosus        0.000000000          0.000000000 0.0000000000
## Thysanoptera                0.000000000          0.000000000 0.0000000000
## amorphous_detritus          0.039923260          0.010458093 0.0062137379
## animal                      0.005132251          0.001744960 0.0008790064
## cyanobacteria               0.007753672          0.002124065 0.0011923827
## diatom                      0.083270267          0.025153484 0.0147222089
## filamentous                 0.007800924          0.002414425 0.0012194497
## green_algae                 0.027855678          0.007808811 0.0048197756
## plant_material              0.008338627          0.002214108 0.0012892594
##                       amorphous_detritus animal cyanobacteria diatom
## Cricotopus sylvestris                  0      0             0      0
## Hirudinidae                            0      0             0      0
## Limnophora riparia                     0      0             0      0
## Lumbricidae                            0      0             0      0
## Midge indet.                           0      0             0      0
## Naidid                                 0      0             0      0
## Nais spp.                              0      0             0      0
## Nematoda                               0      0             0      0
## Oribatid                               0      0             0      0
## Ostracoda                              0      0             0      0
## Radix balthica                         0      0             0      0
## Simulium vittatum                      0      0             0      0
## Sperchon glandulosus                   0      0             0      0
## Thysanoptera                           0      0             0      0
## amorphous_detritus                     0      0             0      0
## animal                                 0      0             0      0
## cyanobacteria                          0      0             0      0
## diatom                                 0      0             0      0
## filamentous                            0      0             0      0
## green_algae                            0      0             0      0
## plant_material                         0      0             0      0
##                       filamentous green_algae plant_material
## Cricotopus sylvestris           0           0              0
## Hirudinidae                     0           0              0
## Limnophora riparia              0           0              0
## Lumbricidae                     0           0              0
## Midge indet.                    0           0              0
## Naidid                          0           0              0
## Nais spp.                       0           0              0
## Nematoda                        0           0              0
## Oribatid                        0           0              0
## Ostracoda                       0           0              0
## Radix balthica                  0           0              0
## Simulium vittatum               0           0              0
## Sperchon glandulosus            0           0              0
## Thysanoptera                    0           0              0
## amorphous_detritus              0           0              0
## animal                          0           0              0
## cyanobacteria                   0           0              0
```

```
## diatom                          0           0              0
## filamentous                     0           0              0
## green_algae                     0           0              0
## plant_material                  0           0              0
```

```
losses = 0.15 * groups.level$bodymasses^(-0.25)

# growth rates of basal sppecies
growth.rates = rep(NA, dim(groups.level$mat)[1])
growth.rates[colSums(groups.level$mat) == 0] = 0.5

val.mat = fluxing(groups.level$mat,
                  groups.level$biomasses,
                  losses,
                  groups.level$efficiencies,
                  bioms.pref = TRUE,
                  ef.level = "pred")

stability.value(val.mat,
                groups.level$biomasses,
                losses,
                groups.level$efficiencies,
                growth.rates,
                ef.level = "pred")
```

```
## [1] -3.583955
```

Seasonality of energy flux

Here I calculate energy fluxes based on seasonal biomasses, where winter is December - February, spring is March - May, summer is June - August, and fall is September - November. Seasonal dates for the trophic interactions are grouped a little be differently: "first" is Jan - Apr, "second" is May - Aug, and "third" is Sept - Dec, so I use first for winter and spring, second for summer, and third for fall. I estimate seasonal metabolic rates based on average body size and temperature for each respective season.

Important to note that the sampling was not conducted during the same year across the gradient; oh2 and st7 were from 2010-2011, and the remaining from 2011-2012.